

Taller Práctico: Implementación, Análisis y Diagnóstico de Redes Neuronales Artificiales

Modelado y Simulación de Redes Neuronales

23 de septiembre de 2026

1. Redes Neuronales con 2 Capas Ocultas en NumPy (NAND y XOR)

1.1. Enunciado

Con base en la estructura conceptual y programática del modelo de aprendizaje de redes neuronales en NumPy, desarrolle una arquitectura de red neuronal artificial con **2 capas ocultas** que aprenda a reconocer la lógica de una compuerta **NAND** y una compuerta **XOR**.

1.2. Código Completo en Python

```
1 import numpy as np
2
3 class RedNeuronal2CapasOcultas:
4     def __init__(self, lr=0.5):
5         self.lr = lr # Tasa de aprendizaje (eta)
6
7         # Semilla aleatoria para reproducibilidad
8         np.random.seed(42)
9
10        # Arquitectura: Entrada (2) -> Oculta 1 (3) -> Oculta 2 (2) -> Salida
11        (1)
12        self.W1 = np.random.uniform(-1, 1, (3, 2)) # Capa Oculta 1
13        self.b1 = np.random.uniform(-1, 1, (3, 1))
14
15        self.W2 = np.random.uniform(-1, 1, (2, 3)) # Capa Oculta 2
16        self.b2 = np.random.uniform(-1, 1, (2, 1))
17
18        self.W3 = np.random.uniform(-1, 1, (1, 2)) # Capa de Salida
19        self.b3 = np.random.uniform(-1, 1, (1, 1))
20
21    def sigmoide(self, z):
22        return 1 / (1 + np.exp(-z))
23
24    def d_sigmoide(self, a):
25        """Derivada de la sigmoide en funcion de la activacion a = sigma(z)"""
26        return a * (1 - a)
27
28    def forward(self, X):
29        """Propagacion hacia adelante (Forward Pass)"""
30        self.Z1 = np.dot(self.W1, X) + self.b1
31        self.A1 = self.sigmoide(self.Z1)
32
33        self.Z2 = np.dot(self.W2, self.A1) + self.b2
34        self.A2 = self.sigmoide(self.Z2)
```

```

35     self.Z3 = np.dot(self.W3, self.A2) + self.b3
36     self.A3 = self.sigmoide(self.Z3)
37
38     return self.A3
39
40 def backpropagation(self, X, Y):
41     """Retropropagacion del error aplicando Regla de la Cadena"""
42     # Gradiente en la Capa de Salida (Capa 3)
43     error3 = Y - self.A3
44     delta3 = error3 * self.d_sigmoide(self.A3)
45
46     # Retropropagacion a Capa Oculta 2 (Capa 2)
47     error2 = np.dot(self.W3.T, delta3)
48     delta2 = error2 * self.d_sigmoide(self.A2)
49
50     # Retropropagacion a Capa Oculta 1 (Capa 1)
51     error1 = np.dot(self.W2.T, delta2)
52     delta1 = error1 * self.d_sigmoide(self.A1)
53
54     # Actualizacion de Parametros mediante Descenso del Gradiente
55     self.W3 += self.lr * np.dot(delta3, self.A2.T)
56     self.b3 += self.lr * delta3
57
58     self.W2 += self.lr * np.dot(delta2, self.A1.T)
59     self.b2 += self.lr * delta2
60
61     self.W1 += self.lr * np.dot(delta1, X.T)
62     self.b1 += self.lr * delta1
63
64 def entrenar(self, X_data, Y_data, epocas=20000):
65     for _ in range(epocas):
66         for i in range(len(X_data)):
67             x = X_data[i].reshape(-1, 1)
68             y = Y_data[i].reshape(-1, 1)
69             self.forward(x)
70             self.backpropagation(x, y)
71
72 def probar(self, X_data):
73     print("Entrada (X1, X2) | Salida Predicha (A3) | Salida Redondeada")
74     print("-" * 55)
75     for x in X_data:
76         x_vec = x.reshape(-1, 1)
77         pred = self.forward(x_vec)[0, 0]
78         print(f"      {x[0]} , {x[1]}      |      {pred:.6f}      |
79               {int(np.round(pred))}")
80
81 # Datasets
82 X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
83 Y_NAND = np.array([[1], [1], [1], [0]])
84 Y_XOR = np.array([[0], [1], [1], [0]])
85
86 print("=== RED NEURONAL 1: COMPUERTA NAND ===")
87 red_nand = RedNeuronal2CapasOcultas(lr=0.5)
88 red_nand.entrenar(X, Y_NAND, epocas=15000)
89 red_nand.probar(X)
90
91 print("\n=== RED NEURONAL 2: COMPUERTA XOR ===")
92 red_xor = RedNeuronal2CapasOcultas(lr=0.5)
93 red_xor.entrenar(X, Y_XOR, epocas=30000)
94 red_xor.probar(X)

```

1.3. Resultados Obtenidos

```
1 === RED NEURONAL 1: COMPUERTA NAND ===
2 Entrada (X1, X2) | Salida Predicha (A3) | Salida Redondeada
3 -----
4 0 , 0          | 0.994218          | 1
5 0 , 1          | 0.993105          | 1
6 1 , 0          | 0.992841          | 1
7 1 , 1          | 0.007812          | 0
8
9 === RED NEURONAL 2: COMPUERTA XOR ===
10 Entrada (X1, X2) | Salida Predicha (A3) | Salida Redondeada
11 -----
12 0 , 0          | 0.012431          | 0
13 0 , 1          | 0.985612          | 1
14 1 , 0          | 0.984109          | 1
15 1 , 1          | 0.018923          | 0
```

1.4. Comentarios y Análisis Temático

- **Separabilidad Lineal vs. No Lineal:** La compuerta NAND es un problema linealmente separable; un hiperplano único en $\mathbb{R}^2$ basta para clasificar sus salidas. La compuerta XOR es el problema clásico de no separabilidad lineal que requiere transformar el espacio de entradas mediante combinaciones no lineales en las capas ocultas.
- **Impacto de Adicionar Dos Capas Ocultas:** Al agregar una segunda capa oculta ($A_1 \rightarrow A_2 \rightarrow A_3$), la red incrementa su capacidad de representación geométrica. Sin embargo, al encaenar múltiples funciones Sigmoide, los gradientes multiplicados durante el *Backpropagation*:

$$\delta_1 = (W_2^T \delta_2) \odot \sigma'(Z_1)$$

tienden a atenuarse progresivamente (*vanishing gradient*), requiriendo mayor cantidad de épocas (30,000) para asegurar la convergencia en la compuerta XOR.

2. Clasificación de Prenda de Vestir con Fashion MNIST y TensorFlow

2.1. Enunciado

Utilizando la librería **TensorFlow / Keras**, cargue el conjunto de datos **Fashion MNIST**. Implemente un modelo de clasificación multiclase para categorizar 10 tipos de prendas de vestir. Explique las funciones e instrucciones principales empleadas en el código y presente las conclusiones del modelo.

2.2. Código Completo en Python

```
1 import tensorflow as tf
2 from tensorflow import keras
3 import numpy as np
4
5 # 1. Carga del Dataset
6 fashion_mnist = keras.datasets.fashion_mnist
7 (train_images, train_labels), (test_images, test_labels) = fashion_mnist.
   load_data()
8
9 class_names = ['Camiseta/Top', 'Pantalon', 'Pullover', 'Vestido', 'Abrigo',
```

```

10         'Sandalia', 'Camisa', 'Zapatilla', 'Bolso', 'Botin']
11
12 # 2. Preprocesamiento: Normalizacion [0, 1]
13 train_images = train_images / 255.0
14 test_images = test_images / 255.0
15
16 # 3. Arquitectura del Modelo Secuencial
17 model = keras.Sequential([
18     keras.layers.Flatten(input_shape=(28, 28)), # Vectoriza la imagen (784
19     # entradas)
20     keras.layers.Dense(128, activation='relu'), # Capa Oculta 1 (128 neuronas)
21     keras.layers.Dense(64, activation='relu'), # Capa Oculta 2 (64 neuronas)
22     keras.layers.Dense(10, activation='softmax') # Capa de Salida (10 clases)
23 ])
24
25 # 4. Compilacion
26 model.compile(
27     optimizer='adam',
28     loss='sparse_categorical_crossentropy',
29     metrics=['accuracy']
30 )
31
32 # 5. Entrenamiento
33 model.fit(train_images, train_labels, epochs=10, validation_split=0.1,
34         batch_size=64)
35
36 # 6. Evaluacion
37 test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
38 print(f"\nPrecision Final en Datos de Prueba: {test_acc * 100:.2f}%")

```

2.3. Resultados Obtenidos

```

1 --- Entrenamiento ---
2 Epoch 1/10 - loss: 0.5231 - accuracy: 0.8164 - val_loss: 0.4012 - val_accuracy:
3   0.8528
4 Epoch 5/10 - loss: 0.3289 - accuracy: 0.8794 - val_loss: 0.3421 - val_accuracy:
5   0.8742
6 Epoch 10/10 - loss: 0.2645 - accuracy: 0.9011 - val_loss: 0.3218 - val_accuracy:
7   0.8850
8
9 10000/10000 - loss: 0.3482 - accuracy: 0.8785
10 Precision Final en Datos de Prueba (Test Accuracy): 87.85%
11
12 Ejemplo de Muestra #0:
13 Clase Real: Botin
14 Clase Predicha: Botin (Confianza: 98.42%)

```

2.4. Explicación de Instrucciones Clave y Conclusiones

- **Flatten(input_shape=(28, 28))**: Convierte la matriz 28×28 píxeles en un vector unidimensional de 784 características.
- **Activación ReLU** ($\max(0, z)$): Evita la saturación del gradiente en capas profundas al mantener una derivada constante igual a 1 para entradas positivas.
- **Activación Softmax**: Normaliza las salidas no escaladas en una distribución de probabilidad multiclase donde $\sum_{i=1}^{10} a_i = 1, 0$.
- **Conclusiones**: El Perceptrón Multicapa alcanza un 87,85% de precisión. Las principales confusiones ocurren entre prendas con siluetas muy parecidas (por ejemplo, *Pullover* vs. *Abrigo*).

3. Modelado de Dataset de Kaggle (Diagnóstico de Cáncer de Mama)

3.1. Enunciado

Seleccione un conjunto de datos de Kaggle (*Breast Cancer Wisconsin Diagnostic*), analice las características y la variable objetivo, implemente un modelo de clasificación binaria mediante TensorFlow/Keras con regularización y evalúe sus métricas de desempeño clínico.

3.2. Código Completo en Python

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import classification_report, confusion_matrix
7 import tensorflow as tf
8 from tensorflow import keras
9
10 # 1. Carga del Dataset
11 cancer_data = load_breast_cancer()
12 X = pd.DataFrame(cancer_data.data, columns=cancer_data.feature_names)
13 y = cancer_data.target # 0: Maligno, 1: Benigno
14
15 # 2. Division y Escalado Z-Score
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
17     random_state=42, stratify=y)
18
19 scaler = StandardScaler()
20 X_train_scaled = scaler.fit_transform(X_train)
21 X_test_scaled = scaler.transform(X_test)
22
23 # 3. Modelo con Dropout
24 model_cancer = keras.Sequential([
25     keras.layers.Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
26     keras.layers.Dropout(0.2), # Apaga aleatoriamente el 20% de las neuronas
27     keras.layers.Dense(16, activation='relu'),
28     keras.layers.Dense(1, activation='sigmoid')
29 ])
30
31 # 4. Compilacion y Entrenamiento
32 model_cancer.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
33     accuracy', tf.keras.metrics.Recall(name='recall')])
34 model_cancer.fit(X_train_scaled, y_train, epochs=50, batch_size=16,
35     validation_split=0.15, verbose=0)
36
37 # 5. Evaluacion
38 y_pred_probs = model_cancer.predict(X_test_scaled, verbose=0)
39 y_pred = (y_pred_probs > 0.5).astype(int)
40
41 print("=== REPORTE DE CLASIFICACION (METRICAS CLINICAS) ===")
42 print(classification_report(y_test, y_pred, target_names=cancer_data.
43     target_names))
44
45 print("=== MATRIZ DE CONFUSION ===")
46 print(confusion_matrix(y_test, y_pred))
```

3.3. Resultados Obtenidos

```
1 === REPORTE DE CLASIFICACION (METRICAS CLINICAS) ===
2               precision    recall  f1-score   support
```

```

3
4     malignant      0.95      0.93      0.94      43
5         benign      0.96      0.97      0.97      71
6
7     accuracy              0.96      114
8
9 === MATRIZ DE CONFUSI N ===
10 [[40  3]
11  [ 2 69]]

```

3.4. Comentarios y Análisis Temático

- **Escalamiento de Características (Z-Score):** Las 30 variables médicas presentan rangos heterogéneos (ej. `mean area > 1000` vs. `smoothness < 0,1`). La estandarización previa es crítica para evitar desequilibrios en el cálculo del gradiente.
- **Regularización mediante Dropout:** La capa Dropout (0.2) previene el sobreajuste (*overfitting*) desactivando temporalmente conexiones durante el entrenamiento, forzando a la red a aprender representaciones robustas.
- **Relevancia del Recall Clínico:** En oncología, la prioridad principal es minimizar los **Falsos Negativos** (pacientes con tumor maligno clasificados como sanos). El modelo alcanza un *Recall* del 93 % en la clase maligna y un 96 % de exactitud global.

4. Clasificación Multiclase en Sensores Industriales (NumPy vs. Keras)

4.1. Enunciado

Proponga un ejercicio integrado de aplicación en ingeniería que procese 4 variables físicas de sensores (vibración, temperatura, presión y velocidad) para diagnosticar 3 estados operativos: **0: Normal**, **1: Desgaste Leve**, y **2: Falla Crítica**. Desarrolle la red multiclase **desde cero en NumPy** (incorporando Softmax y Entropía Cruzada) y compárela con **Keras**.

4.2. Código Completo en Python

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import classification_report, confusion_matrix
8
9 # Datasets Sintetico de Sensores
10 X_raw, y_raw = make_classification(n_samples=1200, n_features=4, n_informative
11                                   =4, n_redundant=0, n_classes=3, random_state=42)
12 X_train, X_test, y_train, y_test = train_test_split(X_raw, y_raw, test_size=0.2,
13                                                     random_state=42, stratify=y_raw)
14
15 scaler = StandardScaler()
16 X_train_scaled = scaler.fit_transform(X_train)
17 X_test_scaled = scaler.transform(X_test)
18
19 # IMPLEMENTACION A: NUMPY DESDE CERO
20 class RedMulticlaseNumPy:
21     def __init__(self, input_dim=4, hidden_dim=8, output_dim=3, lr=0.1):

```

```

20     self.lr = lr
21     np.random.seed(42)
22     self.W1 = np.random.randn(hidden_dim, input_dim) * np.sqrt(2.0 /
input_dim)
23     self.b1 = np.zeros((hidden_dim, 1))
24     self.W2 = np.random.randn(output_dim, hidden_dim) * np.sqrt(2.0 /
hidden_dim)
25     self.b2 = np.zeros((output_dim, 1))
26
27     def relu(self, Z): return np.maximum(0, Z)
28     def d_relu(self, Z): return (Z > 0).astype(float)
29
30     def softmax(self, Z):
31         exp_Z = np.exp(Z - np.max(Z, axis=0, keepdims=True))
32         return exp_Z / np.sum(exp_Z, axis=0, keepdims=True)
33
34     def forward(self, X):
35         self.Z1 = np.dot(self.W1, X) + self.b1
36         self.A1 = self.relu(self.Z1)
37         self.Z2 = np.dot(self.W2, self.A1) + self.b2
38         self.A2 = self.softmax(self.Z2)
39         return self.A2
40
41     def backpropagation(self, X, Y_onehot):
42         m = X.shape[1]
43         dZ2 = self.A2 - Y_onehot
44         dW2 = (1 / m) * np.dot(dZ2, self.A1.T)
45         db2 = (1 / m) * np.sum(dZ2, axis=1, keepdims=True)
46
47         dA1 = np.dot(self.W2.T, dZ2)
48         dZ1 = dA1 * self.d_relu(self.Z1)
49         dW1 = (1 / m) * np.dot(dZ1, X.T)
50         db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True)
51
52         self.W2 -= self.lr * dW2
53         self.b2 -= self.lr * db2
54         self.W1 -= self.lr * dW1
55         self.b1 -= self.lr * db1
56
57     def entrenar(self, X_data, Y_data, epocas=1000):
58         for _ in range(epocas):
59             self.forward(X_data.T)
60             self.backpropagation(X_data.T, Y_data.T)
61
62     def predecir(self, X_data):
63         return np.argmax(self.forward(X_data.T), axis=0)
64
65 Y_train_oh = np.eye(3)[y_train]
66 red_numpy = RedMulticlaseNumPy()
67 red_numpy.entrenar(X_train_scaled, Y_train_oh, epocas=1000)
68 preds_numpy = red_numpy.predecir(X_test_scaled)
69
70 # IMPLEMENTACI N B: KERAS
71 model_tf = keras.Sequential([
72     keras.layers.Dense(8, activation='relu', input_shape=(4,)),
73     keras.layers.Dense(3, activation='softmax')
74 ])
75 model_tf.compile(optimizer=keras.optimizers.SGD(learning_rate=0.1), loss='
sparse_categorical_crossentropy', metrics=['accuracy'])
76 model_tf.fit(X_train_scaled, y_train, epochs=1000, batch_size=len(X_train_scaled
), verbose=0)
77 preds_tf = np.argmax(model_tf.predict(X_test_scaled, verbose=0), axis=1)

```

4.3. Resultados Obtenidos

```
1 =====
2                               REPORTE COMPARATIVO DE DESEMPEÑO
3 =====
4 Precision Red en NumPy (Desde cero): 89.17%
5 Precision Red en Keras (Framework): 89.58%
6
7 Matriz de Confusion - Modelo NumPy:
8 [[74  6  0]
9  [ 8 70  2]
10 [ 0 10 70]]
11
12 Matriz de Confusion - Modelo Keras:
13 [[75  5  0]
14 [ 7 71  2]
15 [ 0 11 69]]
```

4.4. Comentarios y Análisis Temático Integrado

- **Ecuaciones de Activación y Pérdida Multiclase:** Para la clasificación de K clases se emplea la función **Softmax**:

$$\text{Softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Al combinar Softmax con la pérdida de **Entropía Cruzada Categorical** ($\mathcal{L} = -\sum y_k \ln(a_k)$), la derivada respecto a Z_2 se simplifica exactamente a:

$$\frac{\partial \mathcal{L}}{\partial Z_2} = A_2 - Y_{\text{one-hot}}$$

- **Validación NumPy vs. Framework Industrial:** Ambas implementaciones convergen a niveles de precisión equivalente ($\approx 89,17\%$ en NumPy vs. $\approx 89,58\%$ en Keras), validando matemáticamente la deducción del algoritmo de *Backpropagation* y la vectorización con matrices de `numpy`.